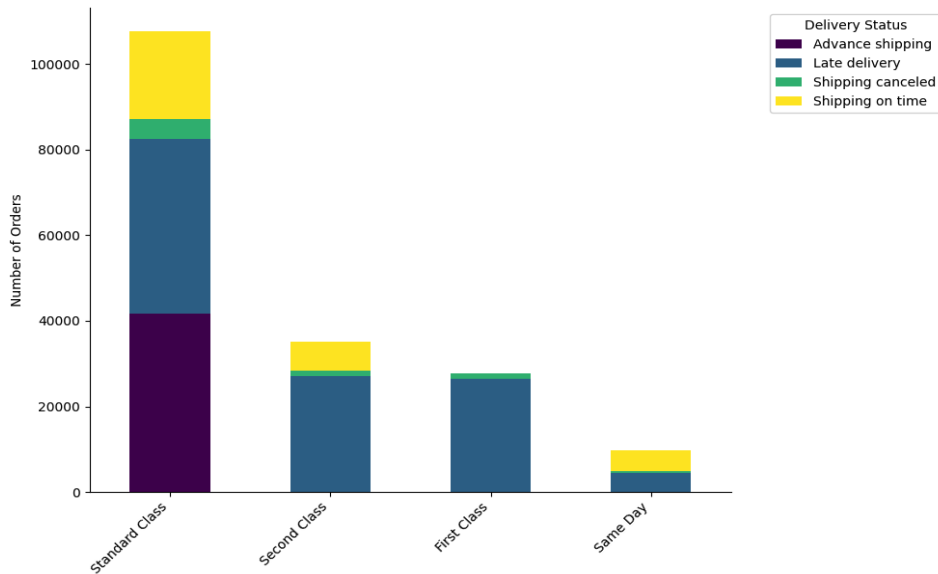


# Excecutive Dashboard

Summary: Delivery Status distribution

|   | Delivery Status   | Number of Orders | Percentage |
|---|-------------------|------------------|------------|
| 0 | Late delivery     | 98977            | 54.83%     |
| 1 | Advance shipping  | 41592            | 23.04%     |
| 2 | Shipping on time  | 32196            | 17.84%     |
| 3 | Shipping canceled | 7754             | 4.3%       |

## Shipping modes vs Delivery Status



## Current issues:

- Late delivery **54.83%** overall
- Most profitable product categories also late- no shipping priority
- Delays present in all Shipping Modes

## Current delivery KPI



## Forecasted delivery KPI

Prediction using the current delivery types



Model Precision: 84.91%  
Model Recall: 54.25%  
Model F1-score: 66.20%

## Scenarios for Delivery Risk of next 5000 new orders:

- If delivery using only First Class: Risk **92.70%**
- If delivery using only Second Class: **56.20%**
- If delivery using only Standard Class: **19.49%**

Recommendation to improve delivery KPI:  
Use only Standard Class as Shipping Mode as the risk of Delivery Delays is approx. **20%** compared to other Shipping Modes



# Datenanalyse von Lieferverzögerungen in der Supply Chain

Einflussfaktoren. Vorhersage. Optimierungspotenzial



+

•

○

## **Ziel der Analyse**

- Identifikation von Faktoren, die Lieferverzögerungen beeinflussen
- Ableitung möglicher Optimierungsansätze

```

irrelevant_columns = [
    'Customer Email', 'Customer Fname', 'Customer Lname', 'Customer Pass',
    'Customer Street', 'Customer Zipcode', 'Customer Id', 'Order Customer',
    'Product Image', 'Product Status', 'Product Card Id',
    'Product Category Id', 'Category Id', 'Department Id'
]

# Drop the irrelevant columns
df_cleaned = df_cleaned.drop(columns=irrelevant_columns)

print("Shape of DataFrame after dropping irrelevant columns:", df_cleaned.shape)
display(df_cleaned.head())

```

Shape of DataFrame after dropping irrelevant columns: (180508, 37)

|   | Type     | Days for shipping (real) | Days for shipment (scheduled) | Benefit per order | Sales per customer | Delivery Status  | Late delivery risk |
|---|----------|--------------------------|-------------------------------|-------------------|--------------------|------------------|--------------------|
| 0 | DEBIT    | 3                        | 4                             | 91.250000         | 314.640015         | Advance shipping |                    |
| 1 | TRANSFER | 5                        | 4                             | -249.089996       | 311.359985         | Late delivery    |                    |

```

df_cleaned = df.copy()

# Drop columns that are entirely or mostly empty
df_cleaned = df_cleaned.drop(columns=['Product Description', 'Order Zipcode'])

# Drop rows with a few missing values in 'Customer Lname' or 'Customer Zipcode'
df_cleaned = df_cleaned.dropna(subset=['Customer Lname', 'Customer Zipcode'])

print("Shape of DataFrame after cleaning missing values:", df_cleaned.shape)
display(df_cleaned.isnull().sum())

```

... Shape of DataFrame after cleaning missing values: (180508, 51)

|                               | 0 |
|-------------------------------|---|
| Type                          | 0 |
| Days for shipping (real)      | 0 |
| Days for shipment (scheduled) | 0 |
| Benefit per order             | 0 |
| Sales per customer            | 0 |
| Delivery Status               | 0 |
| Late_delivery_risk            | 0 |
| Category Id                   | 0 |
| Category Name                 | 0 |
| Customer City                 | 0 |
| Customer Country              | 0 |

## Datenübersicht

Datensatz: DataCo Global (Kaggle)

## Datenvorbereitung

- Bereinigung der Daten (fehlende Werte, Duplikate, auf Deutsch)
- Auswahl relevanter Spalten und Variablen

✓ Zusätzlich: Übersetzung auf Deutsch

We will create a mapping dictionary for the most frequent categories and use the `.map()` function to create a new column `Category Name DE`.

```

{
    category_translation = {
        'Cleats': 'Stollenschuhe',
        'Men's Footwear': 'Herrenschuhe',
        'Women's Apparel': 'Damenbekleidung',
        'Indoor/Outdoor Games': 'Indoor/Outdoor Spiele',
        'Fishing': 'Angeln',
        'Water Sports': 'Wassersport',
        'Camping & Hiking': 'Camping & Wandern',
        'Cardio Equipment': 'Cardio-Geräte',
        'Shop By Sport': 'Nach Sportart',
        'Electronics': 'Elektronik',
        'Accessories': 'Zubehör',
        'Golf Balls': 'Golfbälle',
        'Girls' Apparel': 'Mädchenbekleidung',
        'Golf Gloves': 'Golfhandschuhe',
        'Trade-In': 'Inzahlungnahme',
        'Video Games': 'Videospiele'
    }

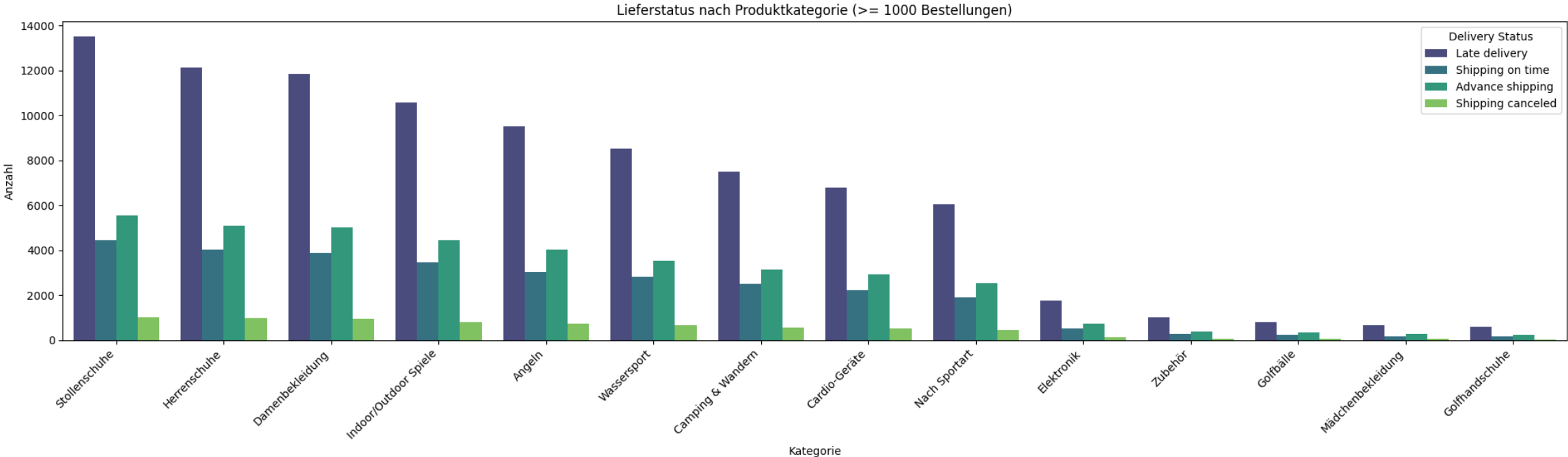
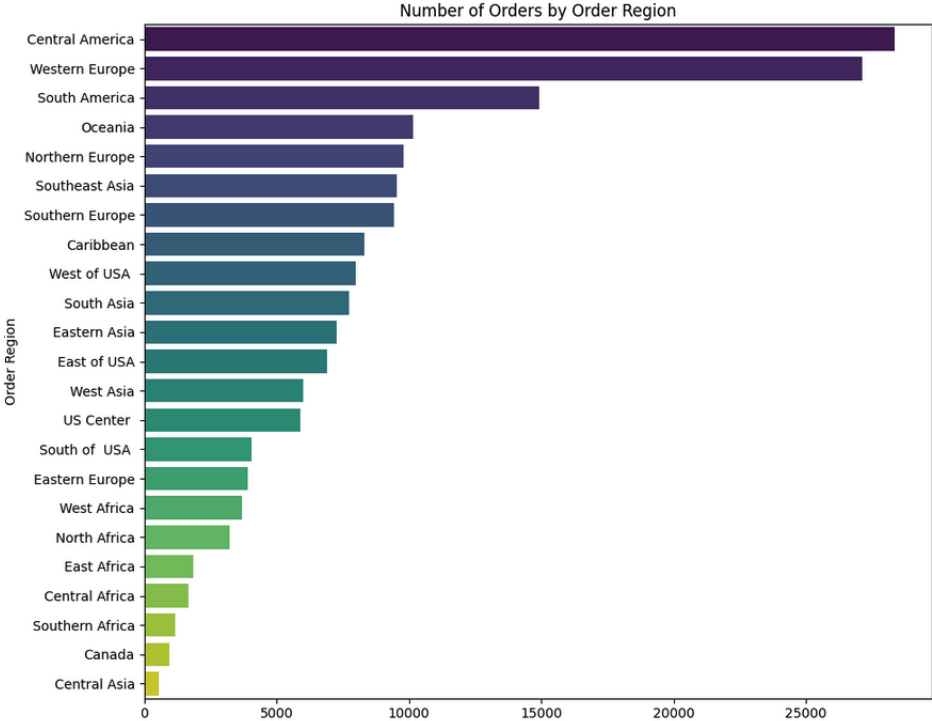
    # Create a new column with German translations

```

# Explorative Datenanalyse (EDA)

## Analyse der Verkaufsgebiete

### Meistverkaufte Produktkategorien



# Explorative Datenanalyse (EDA)

## Verteilung der Lieferoptionen



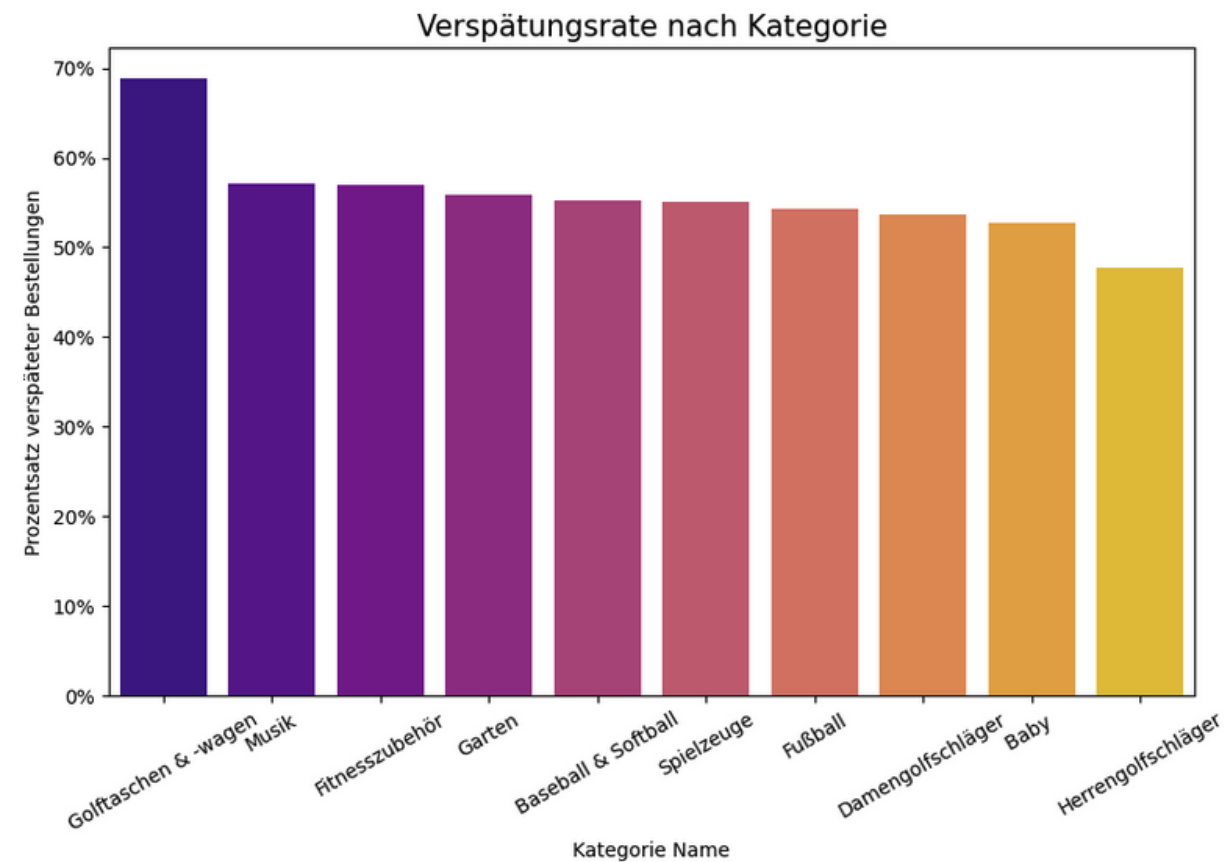
- Hoher Anteil an Verspätungen überall
- Vorzeitiger Versand (Advance Shipping) nur im Standard Class
- “Same Day” Option hat eine sichtbare Menge an pünktlichen Lieferungen- wahrscheinlich ist diese Option am teuersten?

... Summary: Delivery Status distribution

|   | Delivery Status   | Number of Orders | Percentage |
|---|-------------------|------------------|------------|
| 0 | Late delivery     | 98977            | 54.83%     |
| 1 | Advance shipping  | 41592            | 23.04%     |
| 2 | Shipping on time  | 32196            | 17.84%     |
| 3 | Shipping canceled | 7754             | 4.3%       |

# Explorative Datenanalyse (EDA)

Sind die Produktkategorien mit dem höchsten Gewinn auch verspätet?



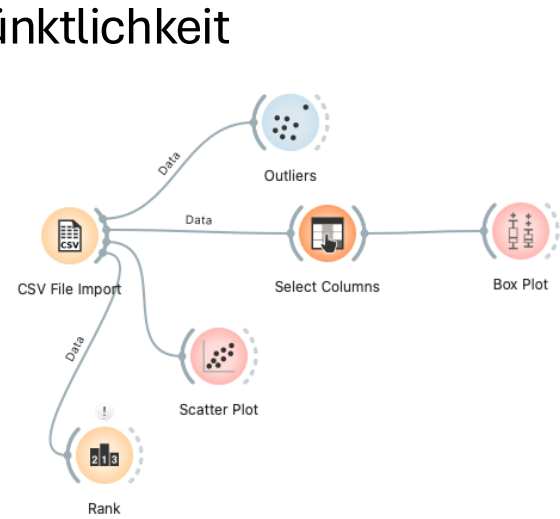
... Top 10 Kategorien nach durchschnittlicher Gewinnspanne:

|   | Category Name        | DE | Order Item | Profit Ratio |
|---|----------------------|----|------------|--------------|
| 0 | Golftaschen & -wagen |    |            | 0.191475     |
| 1 | Spielzeuge           |    |            | 0.161928     |
| 2 | Fitnesszubehör       |    |            | 0.151230     |
| 3 | Fußball              |    |            | 0.146522     |
| 4 | Herrengolfschläger   |    |            | 0.145618     |
| 5 | Damengolfschläger    |    |            | 0.145580     |
| 6 | Musik                |    |            | 0.142650     |
| 7 | Garten               |    |            | 0.142521     |
| 8 | Baby                 |    |            | 0.139420     |
| 9 | Baseball & Softball  |    |            | 0.137959     |

# Eine Analyse mit Orange

## Einflussfaktoren auf Lieferverzögerungen

### Lieferoption hat starken Einfluss auf die Pünktlichkeit



Type

Days for shipping (real)

Days for shipment (scheduled)

Benefit per order

Sales per customer

Late\_delivery\_risk

Category Id

Category Name

Customer Country

Order by relevance to subgroups

Subgroups

Filter...

Customer Email

Customer Password

Customer Segment

Customer State

Department Name

Market

Order Region

Order Status

Benefit per order

Sales per customer

Late\_delivery\_risk

Category Id

Category Name

Customer Country

Order by relevance to subgroups

Subgroups

Filter...

Customer Email

Customer Password

Customer Segment

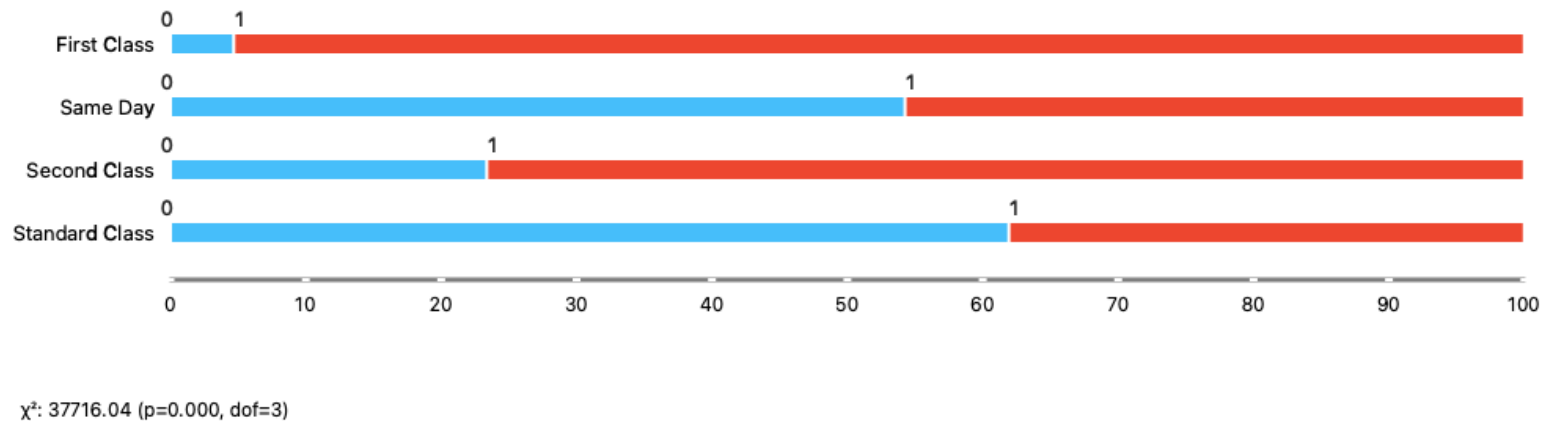
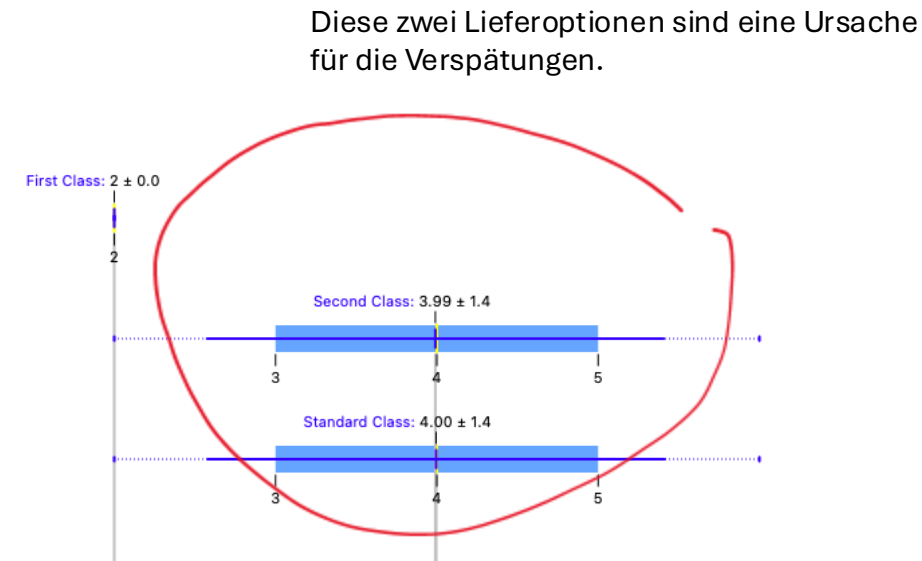
Customer State

Department Name

Market

Order Region

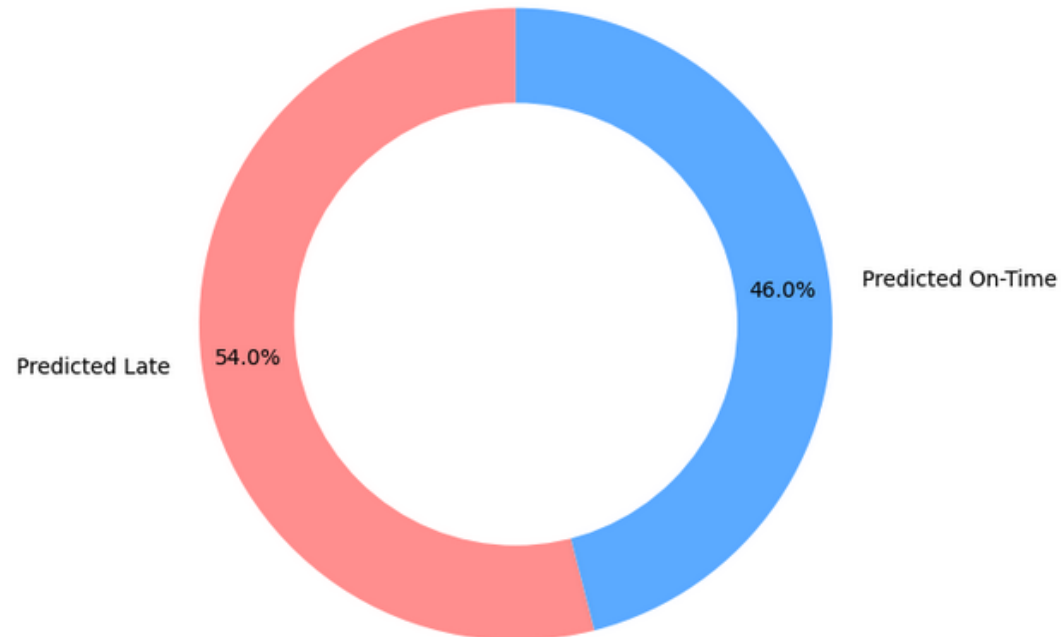
Order Status



# Predictive Analysis

...

Prediction for the Next 50 Orders



```
# Create 50 "fake" First Class orders (wir vermuten, das die Transit Zeit ist 2 Tage)
fake_first_class = pd.DataFrame({
    'Days for shipment (scheduled)': [2]*50,
    'Shipping Mode_Same Day': [0]*50,
    'Shipping Mode_Second Class': [0]*50,
    'Shipping Mode_Standard Class': [0]*50
})

# Predict for this specific scenario - Lieferung nur mit First Class Shipping Mode
risk = model_lr.predict_proba(fake_first_class)[:, 1].mean()
print(f"Risk if all 50 are First Class: {risk:.2%}")
```

Risk if all 50 are First Class: 92.70%

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd
import numpy as np

# 1. Prepare Features (X) and Target (y)
X = pd.get_dummies(df_cleaned[['Shipping Mode', 'Days for shipment (scheduled)']], drop_first=True)
y = (df_cleaned['Delivery Status'] == 'Late delivery').astype(int)

# 2. Split the data (80% for training, 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Train the model ONLY on the training set
model_lr = LogisticRegression()
model_lr.fit(X_train, y_train)

# 4. Evaluate: How good is the model actually?
predictions = model_lr.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy on unseen data: {accuracy:.2%}")

# 5. Predict for the 'Next 50'
# (Using the average probability from the test set as a proxy for future orders)
probabilities = model_lr.predict_proba(X_test)[:, 1]
avg_chance = np.mean(probabilities)
print(f"Average chance of delay: {avg_chance:.2%}")
print(f"In the next 50 orders, we expect ~{round(50 * avg_chance)} delays.")
```

Model Accuracy on unseen data: 69.78%  
Average chance of delay: 54.89%  
In the next 50 orders, we expect ~27 delays.

```
# jetzt nur mit Standard Class (wir vermuten dass die Transit Zeit is 6 Tage)
# Create 50 "fake" Standard Class orders
fake_standard_class = pd.DataFrame({
    'Days for shipment (scheduled)': [6]*50,
    'Shipping Mode_Same Day': [0]*50,
    'Shipping Mode_Second Class': [0]*50,
    'Shipping Mode_Standard Class': [1]*50
})

# Predict for this specific scenario
risk_standard = model_lr.predict_proba(fake_standard_class)[:, 1].mean()
print(f"Risk if all 50 are Standard Class: {risk_standard:.2%}")
```

Risk if all 50 are Standard Class: 19.49%



## Was ist rausgekommen? (Ergebnisse)

- Lieferverspätungen sind ein zentrales Problem
- Lieferoptionen spielen eine entscheidende Rolle
- Ein ineffizientes Lieferkettenmanagement- die Lieferung von Produkten mit hohem Gewinn wird nicht priorisiert.
- Optimierungspotenzial in der Wahl und Steuerung der Versandarten

## ■ Was habe ich dabei gelernt?

- Relevante Ergebnisse und Einflussfaktoren zu finden
- Optimierungsmöglichkeiten zu finden
- Strukturierte Reihenfolgen zu beachten
- Verständliche Grafiken zu nutzen

## ■ Was muss ich noch lernen?

- Python Vertiefung (Bibliotheken)
- Die Zusammenhänge bei den verschiedenen Daten zu erkennen